

객체지향의 3요소

캡슐화, 상속, 다형성

1. 캡슐화

- 객체의 상태(state)와 동작(action)을 하나의 클래스(class)로 묶는 것

* 은닉화

- 외부에 감추고 싶은 상태와 동작은 감추고, 공개하고 싶은 상태와 동작만 공개하는 것

```
class Car {  
    private int speed;  
    private void accelerate() {}  
    public void drive() {  
        System.out.println("Car-drive");  
    }  
}
```

2. 상속

- 부모의 상태와 동작을 자식이 물려받는 것

```
class Genesis extends Car {  
    public void drive() {  
        System.out.println("Genesis-drive");  
    }  
}
```

- 공개되지 않은 상태와 동작은 외부에서 호출할 수 없음

```
public class Test {  
    public static void main(String[] args) {  
        Car c = new Genesis();  
        c.accelerate();  
    }  
}
```

=> 위의 코드는 에러 발생

- 부모 타입의 변수로 자식 인스턴스를 할당할 수 있음

```
Car c = new Genesis();  
Genesis myGen = (Genesis) c;
```

- 자식 타입의 변수로 부모 인스턴스를 할당할 수 없음

```
Genesis myGen = new Car();
```

=> 위의 코드는 에러 발생

- 아래의 코드의 출력 값은?

=> Genesis-drive

```
Car c = new Genesis();c.drive();
```

3. 다형성

- 오버로딩, 오버라이딩, 제네릭

- 오버로딩

- 이름은 같으나 인자의 개수와 인자의 타입이 다른 메소드를 정의하는 것

```
void m() {}
```

```
void m(int a) {} void m(String a) {}
```

```
void m(int a, String b) {}
```

- 오버라이딩

- 부모의 메소드를 자식이 재정의하는 것
- Genesis class의 drive 메소드는 Car class의 drive 메소드를 재정의 하였음

- 제네릭

- 다양한 타입의 상태와 동작을 가질 수 있는 클래스의 틀을 정의하는 것

```
class Car<T> {  
    public T speed;  
}
```

```
Car<Integer> c1 = new Car<>();
```

```
c1.speed = 5;
```

```
Car<String> c2 = new Car<>();
```

```
c2.speed = "빠름";
```

객체지향의 동작 방식

- 객체 간의 메시지 통신

객체지향을 사용하는 이유

- 재사용성 (reusable)

재사용성을 위한 방법

- 아래의 User 클래스를 재사용하고자 할 때, 개인 정보만 필요하다면?

```
class User {  
    개인 정보, 신체 정보, 연애 정보, ..., 등  
}
```

=> 재사용하기 어려움

- 클래스를 간결하게 설계 해야함

```
class User개인정보 {}
```

```
class User신체정보 {}
```

```
class User연애정보 {}
```

추상 클래스와 인터페이스

추상 클래스 (abstract class)의 특징

- 객체를 생성할 수 없음
- 객체가 상태(state)를 가질 수 있음
- 객체의 동작(action) 구현 가능

추상 클래스 (abstract class)의 사용

- ① "is-a" 관계가 성립할 때
 - Bird 클래스는 Animal 클래스의 하위 타입 (Bird is a Animal)
- ② 공통된 상태를 가지거나 동작 구현을 공유해야 할 때

인터페이스 (interface)의 특징

- 추상 메소드만 선언 가능
- 정적 상수만 가능
- 다중 구현 가능

인터페이스 (interface)의 사용

- ① "can-do" 관계가 성립할 때
 - Bird 클래스는 Flyable 인터페이스 구현 가능
- ② 공통적인 동작을 공유하지만, 상태를 가질 필요가 없을 때

```
abstract class Vehicle {
    String name;

    Vehicle(String name) {
        this.name = name;
    }

    abstract void move();
}

interface Electric {
    void chargeBattery();
}

class ElectricCar extends Vehicle implements Electric {
    ElectricCar(String name) {
        super(name);
    }

    @Override
    void move() {
        System.out.println(name + " is driving on the road.");
    }

    @Override
    public void chargeBattery() {
        System.out.println(name + " is charging its battery.");
    }
}
```

디자인 패턴

디자인 패턴 5원칙

1. 단일 책임 원칙 (Single responsibility principle)

- 클래스는 하나의 책임만을 가져야 한다는 원칙
- 클래스를 구현할 때는 한 가지 기능에만 중점을 두어야 함
- 두 가지 이상의 기능이 필요하다면 클래스를 나누어야 함

2. 개방-폐쇄 원칙 (Open-closed principle)

- 소프트웨어 개체(클래스, 모듈, 함수 등)는 확장에 열려(Open)있어야 하고 수정에는 닫혀(Close)있어야 한다는 원칙
- 기본 클래스는 수정하지 않고 클래스를 확장해 새로운 기능을 구현할 수 있어야 함

3. 리스코프 치환 원칙 (Liskov substitution principle)

- 상속 받는 클래스는 기본 클래스의 역할을 완전히 치환할 수 있어야 한다는 원칙
- 파생된 클래스는 기본 클래스를 완전히 확장해야 함
- 상위 클래스는 공통 속성이나 추상화된 기능만을 가지고 있어야 함

4. 인터페이스 분리 원칙 (Interface segregation principle)

- 클래스가 사용하지 않는 인터페이스는 의존하지 않아야 한다는 원칙
- 인터페이스를 구현하는 모든 클래스는 필요 없는 메소드까지 구현해야 하므로 분리해야 한다는 의미

5. 의존 역전 원칙 (Dependency inversion principle)

- 상위 모듈은 하위 모듈에 의존적이지 않아야 한다는 원칙
- 클래스를 참조할 일이 있을 때 직접 참조하지 말고, 추상 클래스를 만들어서 참조해야 함

ex) 아이가 로봇이 아닌 인형을 가지고 놀고 싶어지면?

- 기본 클래스인 Kid 클래스를 변경해야 함
- Kid (상위 모듈) 클래스가 Robot (하위 모듈) 클래스에 의존하고 있음

=> OCP, DIP 위반

```
class Robot {}
class Kid {
    private Robot toy;
    public void play() {}
}
```